Ders 18 Çalışma Özeti

Ders 18 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:06:50'de başlar.
  - Yaklaşık 00:50:00 ile 01:04:15 arası ara verildi.
- Nesneler arasında tip dönüşümü (typecasting)
  - Yukarı dönüşüm (alt sınıftan üst sınıfa) ve aşağı dönüşüm (üst sınıftan alt sınıfa).
  - `instanceof` ile kontrol, `((Food) item).isExpired()` kalıbı.
  - `Calendar.getInstance()` ve fabrika metodu (factory method).
- Dosyalarla çalışma
  - Sıralı (sequential) ve rastgele (random) erişim. Metin (text) ve ikili (binary) format.
  - `IOException`, `EOFException`, `FileNotFoundException`, `SecurityException`.
  - `java.io.File` sınıfı, mutlak (absolute) ve göreli (relative) yol.
  - Akışlar (stream). Serileştirme (serialization) ve ters serileştirme (deserialization).
  - Örnek: telefon rehberi (`Contact`) uygulaması (`nyp12a`, `nyp12b`, `nyp12c` paketleri).
- İç sınıflar (inner class)
  - Bağlı liste (linked list) için `Node` iç sınıfı.
  - `Person` ve `Employee` örnekleri.
- Görsel programlamaya (GUI) giriş
  - C# Windows Forms ile kod yazarak pencere, buton ve metin kutusu oluşturma.
  - Olay işleme (event handling): `MouseMove`, `MouseClick`, `Click`.
  - Ders sonunda Java'da `ActionListener` ve anonim sınıf ile olay bağlama.

Hocanın Özellikle Vurguladığı Kısımlar

- Üst sınıf referansı alt sınıf nesnesini tutabilir (`A a = new B();`). Tersi doğrudan yapılamaz.
  - Örnek: Her öğretmen bir kullanıcıdır. Öğretmen nesnesi kullanıcı referansında tutulabilir.
- Üst sınıftan alt sınıfa dönüşüm risklidir. Önce `instanceof` ile kontrol edilmelidir.
- Tip dönüşümünü gerektiğinde ama az kullanın. Java'nın yeni sürümleri ve soyut sınıflar bu ihtiyacı azaltır.
- **Dosya nasıl yazıldıysa öyle okunur.** Okurken yazılış sırasını ve formatını bilmek gerekir.
- Dosya işlemlerinde mutlaka `try-catch` kullanın.
- Serileştirilecek sınıf `Serializable` arayüzünü uygulamalıdır. Bu arayüzün metodu yoktur. İşaretçi (marker) arayüzdür.
- Okunan nesne `Object` tipinde gelir. Kendi sınıfımıza tip dönüşümü yapmamız gerekir.
- İstisna yakalamayı döngüden çıkmak için kullanmak doğru değildir. Bunun yerine bütün nesneleri bir kapsayıcı (container) nesnede, örneğin `LinkedList` içinde, tek seferde yazın.
- Alıştırma: rehber örneğini dizi yerine `ArrayList` veya `LinkedList` ile yazın. Listeyi tek seferde yazıp tek seferde okuyun.
- GUI'yi sürükle-bırak yerine kodla oluşturmayı öğrenin. Binlerce buton (stadyum, otobüs koltukları) sürükle-bırak ile yönetilemez.

Kısa Tekrar Notları

- `instanceof`: Nesnenin belirli bir sınıftan olup olmadığını `true`/`false` olarak verir.
- Aşağı dönüşüm kalıbı: `((Food) item).isExpired()`.

- Fabrika metodu: Nesneyi `new` yerine statik bir metotla üretmek. Örnek: `Calendar.getInstance()`.
- Sıralı erişim dosyayı baştan sona okur (teyp gibi). Rastgele erişim istenen konuma doğrudan gider.
- İkili format yer kazandırır, okunması zordur. Metin formatı okunur ama daha çok yer kaplar.
- `EOFException` ve `FileNotFoundException`, `IOException`'dan türer. Tek `catch (IOException e)` ikisini de yakalar.
- `new File("deneme.txt")` yalnızca nesne oluşturur, diskte dosya oluşturmaz.
- Windows yolunda ters eğik çizgi Java metninde iki kez yazılır: `"C:\\klasor\\deneme.txt"`.
- Serileştirme: `ObjectOutputStream` + `FileOutputStream`, `writeObject`.
- Ters serileştirme: `ObjectInputStream` + `FileInputStream`, `readObject` + tip dönüşümü.
- `transient` alan serileştirilmez.
- İç sınıf, dış sınıfın `private` üyeleri dahil bütün üyelerine erişebilir.
- `DisSinif.this` ile iç sınıftan dış sınıf nesnesine ulaşılır.
- C#'ta olay bağlama `+=`, Java'da `addActionListener`, PyQt'de `connect` ile yapılır.

Detaylı Açıklamalar

Tip dönüşümü: market rafı örneği

- `Item` bir arayüzdür. `Food` ve `Electronic` bu arayüzü uygular.
- Raf (`Shelf`) bütün ürünleri `LinkedList<Item>` içinde tutar. Arayüzden nesne üretilemez ama arayüz tipinde referans tutulabilir.
- Son kullanma tarihi yalnızca `Food` sınıfında vardır. Bu yüzden önce kontrol, sonra dönüşüm yapılır.

```
public void printExpiredItems() {
    boolean hasExpiredItem = false;
    System.out.println("Expired item(s): ");
    for (Item item : items) {
        if (item instanceof Food)
            if (((Food) item).isExpired()) {
                hasExpiredItem = true;
                System.out.println(item);
            }
    }
    if (hasExpiredItem == false)
        System.out.println("All items are fresh!");
}
```

- `Food.isExpired()`, `Date.before` ile son kullanma tarihini bugünle karşılaştırır.

Calendar ve fabrika metodu

- `Calendar cal = Calendar.getInstance();` statik bir metottur ve bir `Calendar` nesnesi döndürür.
- `cal.add(...)` ile tarihe gün eklenir. `cal.set(...)` ile yıl, ay, gün ayarlanır. `cal.getTime()` bir `Date` döndürür.

Dosya ve dosya biçimi

- Hoca bir PDF dosyasını metin editörüyle açıp bazı baytlarını sildi. Veri kısmını silince dosya kısmen bozuk açıldı. Başlık (header) kısmını bozunca okuyucu dosyayı açamadı.
- Ders: Okuyucu program dosyayı formatına göre bit, bayt veya satır olarak okur. Format bozulursa dosya okunamaz.
- 8 karakterlik bir "0/1" dizisini metin olarak kaydedince dosya 8 bayt olur. Aynı değer ikili olarak 1 bayttır.

File sınıfı ve yollar

- `File(String yol)`, `File(String ust, String ad)`, `File(File ust, String ad)` yapıcıları vardır.
- Faydalı metotlar: `exists()`, `isFile()`, `isDirectory()`, `getParentFile()`, `getCanonicalPath()` (`IOException` fırlatabilir), `canRead()`, `canWrite()`, `createNewFile()`, `mkdir()`, `mkdirs()`, `renameTo()`, `delete()`.
- `File.separator` platforma göre doğru ayıracı verir.

Serileştirme: telefon rehberi

```
public class Contact implements java.io.Serializable {
    private static final long serialVersionUID = 1L;
    private String name, phone, eMail;
    public Contact(String name) { this.name = name; }
    // get/set metotları
    public String toString() {
        return name + " - " + phone + " - " + eMail;
    }
}
```

- `nyp12a`: Önce kayıt sayısı (`Integer`), sonra her `Contact` ayrı ayrı yazılır. Okurken de aynı sırayla okunur.

```
ObjectOutputStream writer =
    new ObjectOutputStream(new FileOutputStream("contacts.dat"));
writer.writeObject(contactCount);
for (Contact c : contacts) writer.writeObject(c);
writer.close();

ObjectInputStream reader =
    new ObjectInputStream(new FileInputStream("contacts.dat"));
int count = (Integer) reader.readObject();
for (int i = 0; i < count; i++) {
    Contact c = (Contact) reader.readObject();
    System.out.println(c);
}
reader.close();
```

- Hoca, ilk okunan nesneyi `Integer` yerine `Contact`'a dönüştürmeyi denedi. Program `ClassCastException` verdi. Sonuç: Dosya nasıl yazıldıysa öyle okunur.
- `contacts.dat` dosyası metin editöründe açılınca `java.lang.Integer`, `files.Contact` gibi sınıf adları ve girilen isimler görünür. Format Java tarafından otomatik oluşturulur.
- `nyp12b`: `LinkedList<Contact>` tek `writeObject` ile yazılır, tek `readObject` ile okunur. Kayıt sayısını ayrıca yazmaya gerek kalmaz.
- `nyp12c`: Dizi (`Contact[]`) de tek seferde yazılıp okunabilir.

İç sınıflar

- Bağlı liste örneği (Sedgewick'in veri yapıları slaytından): `Node` yalnızca bağlı listede kullanılacaksa ayrı dosyada sınıf yazmak yerine liste sınıfının içinde `private` iç sınıf olarak tanımlanabilir.

```
public class LinkedStack<Item> {
    private Node first;
    private class Node {
        Item item;
        Node next;
    }
    public void push(Item item) {
        Node oldFirst = first;
        first = new Node();
```

```
        first.item = item;
        first.next = oldFirst;
    }
}
```

- `Person` içinde `private Employee` iç sınıfı: `main` metodu başka bir sınıfa taşınırsa `Employee` görünmez ve derleme hatası olur.
- `Employee` `public` yapılırsa başka sınıftan `import` ile kullanılabilir.
- Dışarıya açık bir metot (`enlist`) ile `private` iç sınıfın nesnesi dış sınıf içinde oluşturulabilir. `toString`, `employee` alanı `null` değilse maaşı da yazar.
- İç sınıf kalıtım değildir. "Bu sınıf, dış sınıf olmadan anlam taşımaz" ilişkisini kurar.

C# ile GUI ve olay işleme

- `Form` sınıfından kalıtım alan `Pencere` sınıfı yazıldı. Yapıcıda `this.Width`, `this.Height`, `this.Text` ayarlandı.
- Bir GUI nesnesinin dört parçası vardır: adı, özellikleri (property), metotları ve olayları (event).
- Olay bağlama:

```
this.MouseMove += Pencere_MouseMove;
private void Pencere_MouseMove(object sender, MouseEventArgs e) {
    this.Text = "x: " + e.X + " -- y: " + e.Y;
}

Button b1 = new Button();
b1.Location = new Point(50, 50);
b1.Width = 50; b1.Height = 60; b1.Text = "Tıkla";
this.Controls.Add(b1);          // eklenmezse buton görünmez
b1.Click += B1_Click;
```

- Hoca bu bağlamayı C'deki fonksiyon işaretçilerine benzetti. C#'taki karşılığı `delegate`'tir.
- Java'da `button.addActionListener(...)` ile bağlanır. Eski yöntem her olay için ayrı sınıf yazmaktı. Sonra anonim iç sınıflar, daha sonra lambda ifadeleri geldi.

Kalan konular

- Thread, iç sınıflar, birim testi (unit test), Java'da GUI.

Ders sonrası notlar

- Hoca `double d = 3.2; int i = d;` satırında örtük (implicit) dönüşüm olduğunu söyledi. Java'da bu satır derlenmez ("possible lossy conversion"). Açık dönüşüm gerekir: `int i = (int) d;`. Genişletici dönüşüm (`int` → `double`) ise örtüktür.
- "Normalde `Calendar cal = new Calendar()` diyebilirdik" ifadesi yanlış. `Calendar` soyut (abstract) bir sınıftır, `new` ile nesnesi üretilemez. Bu yüzden `getInstance()` kullanılır.
- Göreli yol örneğinde `../YeniKlasor/deneme.txt` için "bulunduğum klasörün altındaki klasör" dendi. `..` bir üst klasör demektir. Bu yol, bir üst klasördeki `YeniKlasor`'u gösterir. Çalışma klasörünün altındaki klasör için `YeniKlasor/deneme.txt` yazılır.
- İkili örnekte `01010001` = 81 değeri için harf olarak "M" dendi. ASCII'de 81 = `Q`, 77 = `M`.
- "Serileştirmede ilkel veri tipi olmamalı" ifadesi kesin değil. `int` gibi ilkel alanlar sorunsuz serileştirilir. `writeObject(5)` çağrısında değer `Integer`'a otomatik kutulanır (autoboxing). İlkel değerler için `writeInt` gibi metotlar da vardır.
- `serialVersionUID` her yazmada artan bir sayaç değildir. Sınıfın sürüm kimliğidir. Okuma sırasında dosyadaki değer sınıftakiyle eşleşmezse `InvalidClassException` oluşur.
- Hoca `catch` sırası için önce `IOException`, sonra `EOFException` yazılabileceğini söyledi. Bu sıra derlenmez: `exception EOFException has already been caught` (Java ile denendi). Önce alt sınıf yazılır.

- Hoca dosya silme girişiminde `FileNotFoundException` alınacağını söyledi. Java ile denendi: Olmayan dosya için `File.delete()` istisna fırlatmadı, `false` döndürdü.
- Hoca işletim sisteminin izin vermediği durumu `SecurityException` ile anlattı. Linux'ta okuma izni kapalı dosya `FileInputStream` ile açılınca `FileNotFoundException` alındı (Java ile denendi).
- Hoca `getCanonicalPath`'in olmayan dosya için `IOException` fırlatabileceğini söyledi. Java ile denendi: Olmayan yol için istisnasız tam yol döndü.
- Hoca `ClassNotFoundException`'ı dönüşüm olmamasıyla ilişkilendirdi. Hatalı dönüşüm `ClassCastException` verir. `ClassNotFoundException` okunan nesnenin sınıfı bulunamayınca oluşur.
- `@SuppressWarnings("unchecked")` istisnalarla ilgili değildir. Jenerik tiplere yapılan denetlenmemiş dönüşüm (unchecked cast) uyarısını gizler. Örnek: `(LinkedList<Contact>) reader.readObject()`.
- `ObjectInputStream` sınıfında hiç `hasNext` metodu olmadı. "JDK 8'den itibaren kaldırıldı" ifadesi doğru değil. Dosya sonuna gelince `readObject` `null` döndürmez, `EOFException` fırlatır.
- Kalıtımda üst sınıf `Serializable` ise alt sınıf da otomatik olarak serileştirilebilir. Alt sınıf `Serializable` olup üst sınıf olmazsa, üst sınıfın alanları yazılmaz ve üst sınıfın parametresiz yapıcısı olmalıdır.
- İç sınıflarda `static` metot tanımlanamayacağı bilgisi Java 15 ve öncesi için doğrudur. Java 16'dan itibaren iç sınıflarda statik üyeler tanımlanabilir.
- Ders sonundaki soru-cevapta iç sınıfların bellek maliyeti konuşuldu. Hoca `static` olmayan iç sınıfın her nesnesinin dış sınıf nesnesine gizli bir referans tuttuğunu anlattı. JDK 25'te bu, dış sınıfı kullanmayan iç sınıflar için geçerli değil: Java ile denendi, dış sınıfı kullanmayan iç sınıfta alan yok (`this$0` üretilmedi), kullanan iç sınıfta bir alan var. Bağlı listedeki `Node` gibi dış nesneye ihtiyaç duymayan sınıflar `private static class Node<Item>` olarak yazılırsa bu referans olmaz. Statik iç sınıf dış sınıfın tip parametresini göremediği için kendi tip parametresini (`<Item>`) alır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.